

规范做事情，则一切尽在掌握中。

提示 1-1：整数值用%d 输出，实数用%f 输出。

这里的“整数值”指的是 $1+2$ 、 $8/5$ 这样“整数之间的运算”。只要运算符的两边都是整数，则运算结果也会是整数。正因为这样， $8/5$ 的值才是 1，而不是 1.6。

8.0 和 5.0 被看作是“实数”，或者说得更专业一点，叫“浮点数”。浮点数之间的运算结果是浮点数，因此 $8.0/5.0=1.6$ 也是浮点数。注意，这里的运算符“/”其实是“多面手”，它既可以做整数除法，又可以做浮点数除法^①。

提示 1-2：整数/整数=整数，浮点数/浮点数=浮点数。

这条规则同样适用于加法、减法和乘法，不过没有除法这么容易出错——毕竟整数乘以整数的结果本来就是整数。

算术表达式可以和数学表达式一样复杂，例如：

程序 1-3 复杂的表达式计算

```
#include<stdio.h>
#include<math.h>
int main()
{
    printf("%.8f\n", 1+2*sqrt(3)/(5-0.1));
    return 0;
}
```

相信读者不难把它翻译成数学表达式 $1 + \frac{2\sqrt{3}}{5-0.1}$ 。尽管如此，读者可能还是有一些疑惑：

5-0.1 的值是什么？“整数-浮点数”是整数还是浮点数？另外，多出来的#include<math.h> 有什么作用？

第 1 个问题相信读者能够“猜到”结果：整数-浮点数=浮点数。但其实这个说法并不准确。确切的说法是：整数先“变”成浮点数，然后浮点数-浮点数=浮点数。

第 2 个问题的答案是：因为程序 1-3 中用到了数学函数 sqrt。数学函数 sqrt(x) 的作用是计算 x 的算术平方根（若不信，可输出 sqrt(9.0) 的值试试）。一般来说，只要在程序中使用到了数学函数，就需要在程序最开始处包含头文件 math.h，并在编译时连接数学库。如果不知道如何编译并运行这段程序，可阅读本章末尾的内容。

1.2 变量及其输入

1.1 节的程序虽好，但有一个遗憾：计算的数据是事先确定的。为了计算 $1+2$ 和 $2+3$ ，

^① 但也有不少语言会严格区分整数除法和浮点数除法。

下面不得不编写两个程序。可不可以让程序读取键盘输入，并根据输入内容计算结果呢？答案是肯定的。程序如下：

程序 1-4 a+b 问题

```
#include<stdio.h>
int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d\n", a+b);
    return 0;
}
```

该程序比 1.1 节的复杂了许多。简单地说，第一条语句“`int a, b`”声明了两个整型（即整数类型）变量 `a` 和 `b`，然后读取键盘输入，并放到 `a` 和 `b` 中。注意 `a` 和 `b` 前面的“&”符号——千万不要漏掉，不信可以试试^①。

现在，你的程序已经读入了两个整数，可以在表达式中自由使用它们，就好比使用 12、597 这样的常数。这样，表达式 `a+b` 就不难理解了。

提示 1-3: `scanf` 中的占位符和变量的数据类型应一一对应，且每个变量前需要加“&”符号。

可以暂时把变量理解成“存放值的场所”，或者形象地认为每个变量都是一个盒子、瓶子或箱子。在 C 语言中，变量有自己的数据类型，例如，`int` 型变量存放整数值，而 `double` 型变量存放浮点数值（专业的说法是“双精度”浮点数）。如果一定要把浮点数值存放在一个 `int` 型变量中，将会丢失部分信息——我们不推荐这样做。

下面来看一个复杂一点的例子。

例题 1-1 圆柱体的表面积

输入底面半径 r 和高 h ，输出圆柱体的表面积，保留 3 位小数，格式见样例。

样例输入：

3.5 9

样例输出：

Area = 274.889

【分析】

圆柱体的表面积由 3 部分组成：上底面积、下底面积和侧面积。由于上下底面积相等，完整的公式可以写成：表面积=底面积 $\times 2$ +侧面积。根据几何知识，底面积= πr^2 ，侧面积= $2\pi rh$ 。不难写出完整程序：

^① 在学习编程时，“明知故犯”是有益的：起码你知道了错误时的现象。这样，当真的不小心犯错时，可以通过现象猜测到可能的原因。

程序 1-5 圆柱体的表面积

```
#include<stdio.h>
#include<math.h>
int main()
{
    const double pi = acos(-1.0);
    double r, h, s1, s2, s;
    scanf("%lf%lf", &r, &h);
    s1 = pi*r*r;
    s2 = 2*pi*r*h;
    s = s1*2.0 + s2;
    printf("Area = %.3f\n", s)
    return 0;
}
```

这是本书中第一个完整的“竞赛题目”，因为和正规比赛一样，题目中包含着输入输出格式规定，还有样例数据。大多数的算法竞赛包含如下一些相同的“游戏规则”。

首先，选手程序的执行是自动完成的，没有人工干预。不要在用户输入之前打印提示信息（例如“Please input n:”），这不仅不会为程序赢得更高的“界面友好分”，反而会让程序丢掉大量的（甚至所有的）分数——这些提示信息会被当作输出数据的一部分。例如，刚才的程序如果加上了“友好提示”，输出信息将变成：

```
Please input n:
Area = 274.889
```

比标准答案多了整整一行！

其次，不要让程序“按任意键退出”（例如，调用 `system("pause")`，或者添加一个多余的 `getchar()`），因为不会有人来“按任意键”的。不少早期的 C 语言教材会建议在程序的最后添加这样一条语句来“观察输出结果”，但注意千万不要在算法竞赛中这样做。

提示 1-4：在算法竞赛中，输入前不要打印提示信息。输出完毕后应立即终止程序，不要等待用户按键，因为输入输出过程都是自动的，没有人工干预。

在一般情况下，你的程序不能直接读取键盘和控制屏幕：不要在算法竞赛中使用 `getch()`、`getche()`、`gotoxy()` 和 `clrscr()` 函数（早期的教材中可能会介绍这些函数）。

提示 1-5：在算法竞赛中不要使用头文件 `conio.h`，包括 `getch()`、`clrscr()` 等函数。

最后，最容易忽略的是输出的格式：在很多情况下，输出格式是非常严格的，多一个或者少一个字符都是不可以的！

提示 1-6：在算法竞赛中，每行输出均应以回车符结束，包括最后一行。除非特别说明，每行的行首不应有空格，但行末通常可以有多个空格。另外，输出的每两个数或者字符串之间应以单个空格隔开。